

Embeddings, Similarity, and Vector Search Notes

Embeddings, cosine similarity, semantic search, vector stores, metadata, top-k, and retrieval debugging.

Engineering notes and worked references

Contents

1. What an embedding represents
2. Embedding dimensions
3. Cosine similarity
4. Semantic search
5. Keyword search and exact identifiers
6. Vector stores
7. Metadata
8. Top-k
9. Similarity thresholds
10. Debugging vector search

1. What an embedding represents

An embedding is a fixed-length numeric vector produced from an input such as text. The useful property is that texts with related meaning often end up closer in the vector space than unrelated texts.

The numbers themselves are not human-readable concepts. You normally compare vectors with a similarity or distance function rather than inspecting individual dimensions.

Example from use

A question about 'changing a reservation time' can be close to a note about 'rescheduling a booking' even when the exact words differ.

The same embedding model should normally be used for stored chunks and incoming queries.

Things people mix up

- Calling an embedding a database.
- Assuming every close vector is factually relevant.
- Comparing vectors from unrelated embedding models as if the dimensions mean the same thing.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with what an embedding represents, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

2. Embedding dimensions

Each embedding model returns vectors of a fixed size. all-MiniLM-L6-v2 returns 384 values per text input.

If 200 chunks are embedded with that model, the result can be viewed as a 200 by 384 matrix: one row for each chunk and one column for each vector dimension.

Example from use

The 384 values are not 384 keywords.

Changing to another embedding model can change the dimension and requires rebuilding the stored vectors.

Things people mix up

- Confusing number of documents with vector dimension.
- Hard-coding 384 everywhere when the model might change.
- Storing the text but forgetting which model produced its vector.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with embedding dimensions, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

3. Cosine similarity

Cosine similarity compares the angle between two vectors. In semantic search it is often used as a measure of directional similarity.

A higher cosine score often indicates more similar meaning for vectors produced by the same embedding model. The number is not a universal probability and thresholds should be tested on the actual data.

Example from use

A score of 0.78 does not mean '78 percent correct'.

Ranking is often more useful than a fixed cutoff for a small knowledge base.

Things people mix up

- Treating similarity as confidence that an answer is true.
- Comparing scores across different embedding models without testing.
- Assuming the highest result must always be relevant.

Quick check

- Can the idea be explained without relying on one memorized definition?

- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with cosine similarity, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

4. Semantic search

Semantic search retrieves text by meaning rather than exact word overlap. It is useful when users paraphrase the source or use different vocabulary.

It works best when the embedded units are coherent enough to represent one topic. If a chunk contains five unrelated sections, the vector can become a blurry average of them.

Example from use

'How do I keep conversation state?' can retrieve a note titled 'Session memory' even without the word keep.

'What is top-k?' can retrieve a section explaining retrieval result count.

Things people mix up

- Expecting semantic search to perfectly handle every exact identifier.
- Using very large mixed-topic chunks.
- Never reviewing the returned text.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with semantic search, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

5. Keyword search and exact identifiers

Embeddings are not a complete replacement for exact matching. Error codes, invoice numbers, file names, API route names, and rare identifiers can be easier to retrieve with keyword or metadata checks.

A practical knowledge system can combine semantic retrieval with exact filters or keyword lookup when the query contains identifiers.

Example from use

A search for ORD-500 should match the exact code before relying on semantic meaning.

A question about '/chat' may benefit from exact route text.

Things people mix up

- Forcing every search through embeddings.
- Assuming a typo in an identifier should match a different identifier.
- Removing original text after embedding it.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with keyword search and exact identifiers, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

6. Vector stores

A vector store keeps vectors and usually some metadata needed to retrieve the original text. Chroma provides a convenient collection-style interface. FAISS focuses on fast vector similarity search.

The vector store is not the embedding model. The model creates the vectors. The store indexes and searches them.

Example from use

The same document can be embedded with one model and stored in Chroma.

FAISS can search vectors while metadata is kept in a separate structure.

Things people mix up

- Saying 'Chroma creates the embeddings' when your application used sentence-transformers.
- Treating a vector index as the original document store.
- Losing the chunk ID needed to return source text.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with vector stores, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

7. Metadata

Metadata lets the application retain file name, section title, page, chunk number, topic, date, or another useful attribute next to the vector.

Good metadata makes debugging easier because a search result can be traced back to the source without guessing which document it came from.

Example from use

A chunk can store `source_file='rag_notes.pdf'`, `section='Top-k'`, `chunk_id='rag-07'`.

Filters can narrow retrieval to one product, subject, or date range when the question requires it.

Things people mix up

- Storing vectors with no source reference.
- Putting large document text inside metadata fields that are meant to stay small.
- Using metadata filters that accidentally exclude the right answer.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with metadata, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

8. Top-k

Top-k is the number of nearest results returned by a retrieval search. A small k can miss supporting context. A large k can add weak matches and consume prompt space.

There is no universal best k. Test the knowledge base with real questions and inspect what is returned.

Example from use

Top-3 may work for a short FAQ where each answer is self-contained.

A question that needs two sections may need more than one retrieved chunk.

Things people mix up

- Assuming bigger k always improves answers.
- Choosing k only by latency without checking coverage.
- Sending all retrieved results to the model without checking relevance.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with top-k, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

9. Similarity thresholds

A threshold can be used to reject results below a chosen similarity level, but the threshold should come from observed retrieval behavior.

Different topics, chunk sizes, and models can produce different score ranges. A hard threshold copied from another project can drop useful results or admit noise.

Example from use

Test supported and unsupported questions and inspect score distributions.

A threshold can be combined with top-k rather than used as the only rule.

Things people mix up

- Treating one threshold as universal.
- Calling low-score retrieval proof that the source does not contain the answer.
- Ignoring exact-match evidence because its embedding score is slightly lower.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with similarity thresholds, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.
- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.

10. Debugging vector search

When retrieval feels wrong, inspect the query text, embedding model, chunk text, metadata, top-k, and returned scores before changing the generation prompt.

A bad answer can begin with a bad search query. Fixing retrieval is easier when the system logs chunk IDs and scores.

Example from use

If the correct section never appears in the results, the problem is before generation.

If the correct section is ranked first but the answer contradicts it, investigate the answer step instead.

Things people mix up

- Changing the LLM when the retriever is the part failing.
- Re-embedding only half the corpus after switching models.
- Debugging from the final answer without printing retrieved chunks.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

How this shows up in a real search

When working with debugging vector search, keep the query text, vector model, result IDs, scores, and source text together during debugging. Looking only at the final answer hides whether the search actually found the right material.

Search quality is affected by several small choices at once: what text was embedded, how large the chunks are, whether metadata filters are active, how many results are returned, and whether exact identifiers need a separate lookup path.

Useful inspection

- Print the top few chunk IDs and short excerpts.
- Compare a direct question with a paraphrase.
- Try one exact identifier or code-like query.

- Test at least one unsupported question.

If two chunks receive similar scores, read them. The ranking number is useful evidence, but the actual text still determines whether the result supports the question.